

Integration of MQTT and Python for Big Data Storage Across SQL, Document and Graph Database Platforms

Project track DB-B1 — Environmental Monitoring

Abebe, Eyasu Belete • Student ID: 571914 • Eyasu_abebe@studenti.unime.it

GitHub: github.com/Joshua-Abebe/environment-monitoring-system

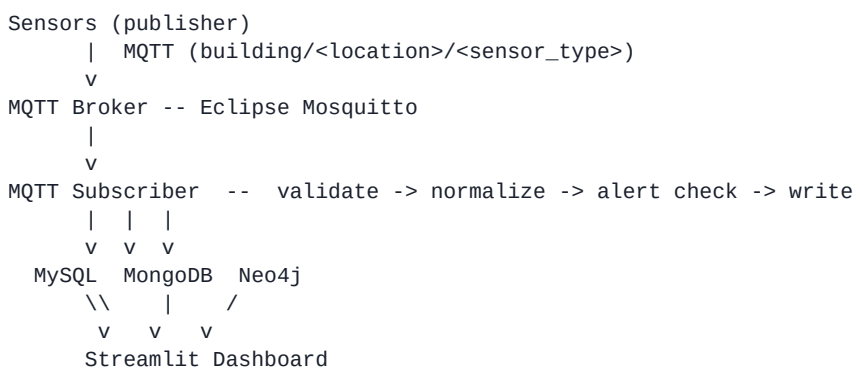
1. Introduction

The growing adoption of IoT sensors has made it common for a single physical environment to produce a continuous stream of heterogeneous data: simple numeric readings, free-form event records, and a web of relationships between devices, locations and the metrics they measure. No single database model handles all three well at once. This project addresses that problem directly: it builds a real-time pipeline that simulates environmental IoT sensors (temperature, humidity, air quality) across several rooms of a building, transports their readings over MQTT, and routes every event into three different database paradigms — a relational store, a document store and a graph store — each used for what it does best, with the results surfaced through a live web dashboard.

The objective was not just to make the pipeline run once, but to make it behave like a small production system: validating and normalizing incoming data defensively, isolating failures so one database having a problem cannot silently take the others down with it, instrumenting every pipeline stage for performance, and covering the core logic with an automated test suite that runs in CI on every change.

2. System Architecture and the MQTT Layer

Data flows in one direction through five stages: simulated sensors publish to an MQTT broker, a single subscriber process consumes every message and fans it out to the three databases, and a dashboard reads back from those databases for display.



Eclipse Mosquitto was chosen as the broker over heavier alternatives such as EMQX: it is lightweight, trivial to containerize, and the project's throughput (a handful of simulated sensors publishing every few seconds) does not need a clustered, enterprise-grade broker. Each sensor publishes to a topic that encodes both its location and measurement type, e.g. `building/lab_a/temperature`, which lets the subscriber and any future consumer filter by either dimension using standard MQTT wildcards (`building/#`). The whole stack — broker, three databases, publisher, subscriber and dashboard — is containerized with Docker Compose, with health checks gating startup order so the subscriber and dashboard never try to connect before their databases are actually ready.

3. Python Application: Publisher and Subscriber

Both sides of the pipeline are built on the Paho MQTT client library.

Publisher — realistic sensor simulation

Six simulated sensors (temperature, humidity and air quality, spread across four rooms) each run on their own thread and publish on their own interval. The first version of the value generator was a plain bounded random walk, which turned out to be a poor simulation of a real sensor: it drifts and then lingers near whichever bound it last wandered to, so it spent a large fraction of its time inside the alert zone and produced an unrealistic, almost constant stream of alerts. This was replaced with a mean-reverting walk (each step pulls the value back toward a configured “normal” baseline) plus a small, independent per-reading probability of a sharp anomaly spike toward one of the bounds. The result is a sensor that idles convincingly around a realistic operating value and only occasionally produces a genuine excursion — alerts happen, but they are the exception rather than the rule, which is what the dashboard's Alerts page is actually meant to show.

Subscriber — validation, normalization and alerting

Every incoming message is parsed as JSON and checked against a required-field schema before anything else happens to it; malformed or incomplete messages are logged and dropped rather than allowed to crash the pipeline. The sensor type string is defensively normalized (trimmed, lower-cased, spaces collapsed to underscores) before it is used in any comparison, so a stray typo or formatting inconsistency in a publisher's payload cannot silently disable an alert threshold check downstream. Alerting itself is edge-triggered: a sensor that stays over threshold across many consecutive readings produces exactly one alert row, the moment it first crosses, rather than one row per reading for as long as the excursion lasts — the alarm state is tracked per sensor and re-arms once the value drops back under threshold.

4. Multi-Database Integration

Rather than picking one database and forcing every kind of data through it, each store was chosen for the access pattern it is actually good at — a deliberate polyglot-persistence design.

Database	Role in this project	Client library
MySQL	Structured relational storage: locations, sensors, readings, alerts, and per-event pipeline performance metrics. Backs the real-time dashboard.	mysqlclient (python)
MongoDB	Archives every raw incoming JSON event exactly as received, with no fixed schema, preserving the original document for forensics.	python-mongo
Neo4j	Models sensors, locations and metrics as a graph (Sensor-LOCATED_IN-Location (Sensor-READS-METRIC, Location-CLOSED-TO-Location).	python-neo4j

A second, equally deliberate decision was resilience between stores: the write to each database is wrapped in its own isolated try/except block rather than one block around the whole pipeline. This was not a hypothetical concern — during testing, a MongoDB container restart caused writes to that one database to fail silently for a period while MySQL and Neo4j kept working normally, and the original single-try/except design would have swallowed that failure along with the rest of the event's processing (including the performance-metrics row). Isolating each database's write means a single store having an outage degrades gracefully — it is logged distinctly and the rest of the pipeline keeps running — instead of taking the whole event down with it.

5. Test and Performance Evaluation

Functional testing

An automated pytest suite (48 tests) covers sensor value generation and bounds, the statistical behaviour of the mean-reverting walk, MQTT event validation, alert-threshold logic (including sensor-type normalization and the edge-triggered debounce), and all three database handlers. Every external client — the MQTT broker, MySQL, MongoDB and Neo4j — is mocked, so the suite runs in seconds without any live infrastructure. A GitHub Actions workflow runs the full suite on every push and pull request across Python 3.11 and 3.12.

Performance evaluation

The dashboard's Performance page times each write stage of the pipeline independently for every event (validation, MySQL write, Neo4j write, MongoDB write, and total latency), so a slowdown in any one database is immediately visible instead of being hidden inside a single end-to-end number. The same page benchmarks

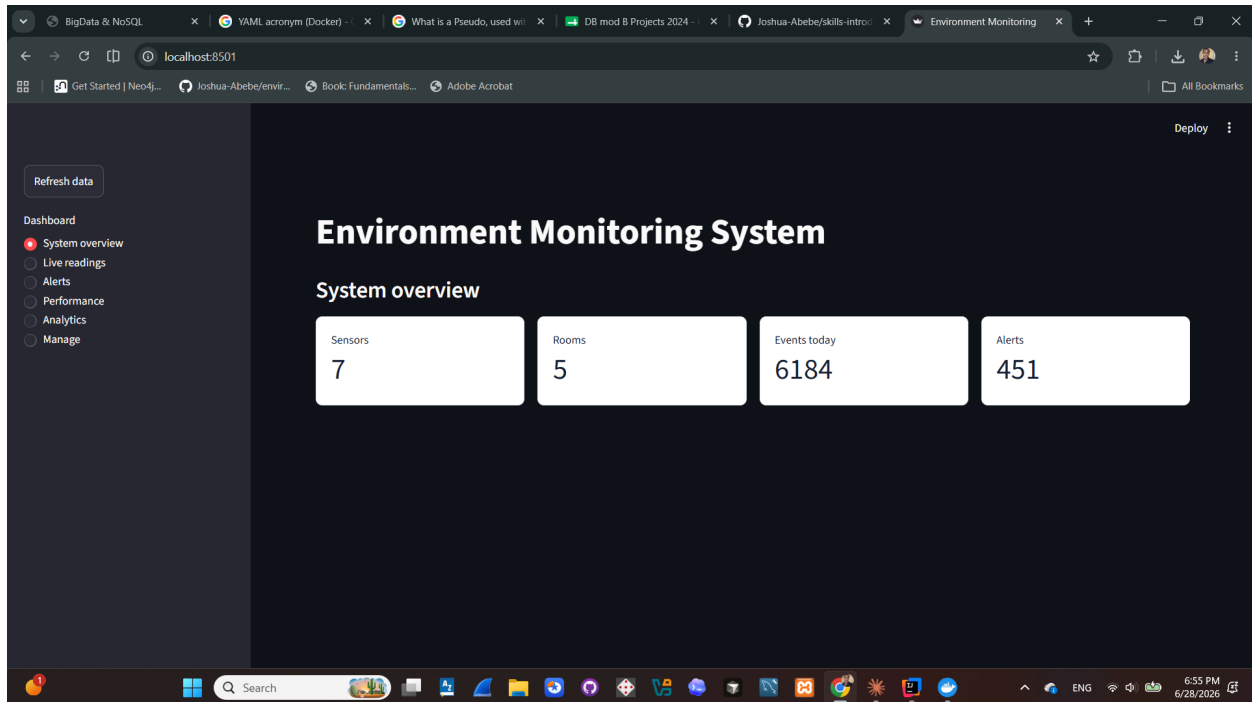
read latency per database; each connection is warmed up with a throwaway call before the timed query runs, which matters in practice — without the warm-up, Neo4j's one-time BOLT handshake (a multi-message connect-and-authenticate exchange) made it look like consistently the slowest database, when in fact its query performance was comparable to the others once connection setup was excluded.

Manual end-to-end verification

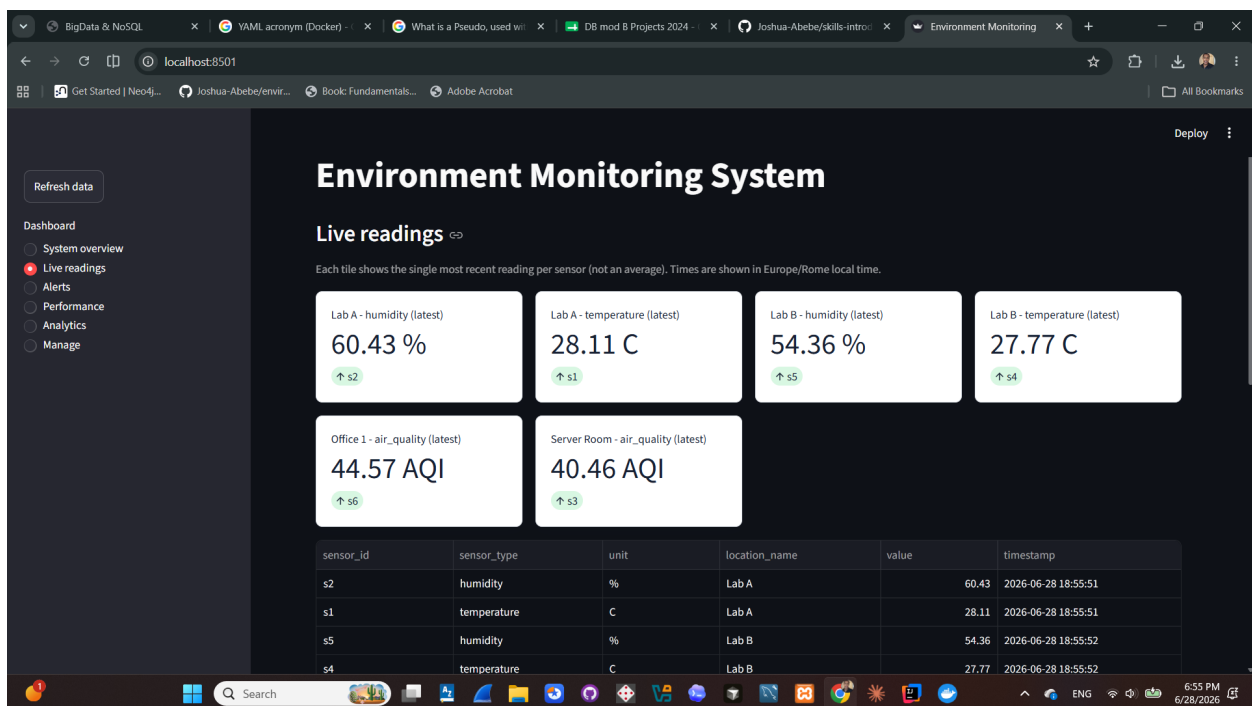
The full stack was run via Docker Compose and exercised manually: live readings, alerts and analytics were confirmed to populate correctly from real traffic, and adding or removing rooms and sensors from the dashboard's Manage page was confirmed to take effect immediately across the relational and graph views without restarting any container.

6. Results

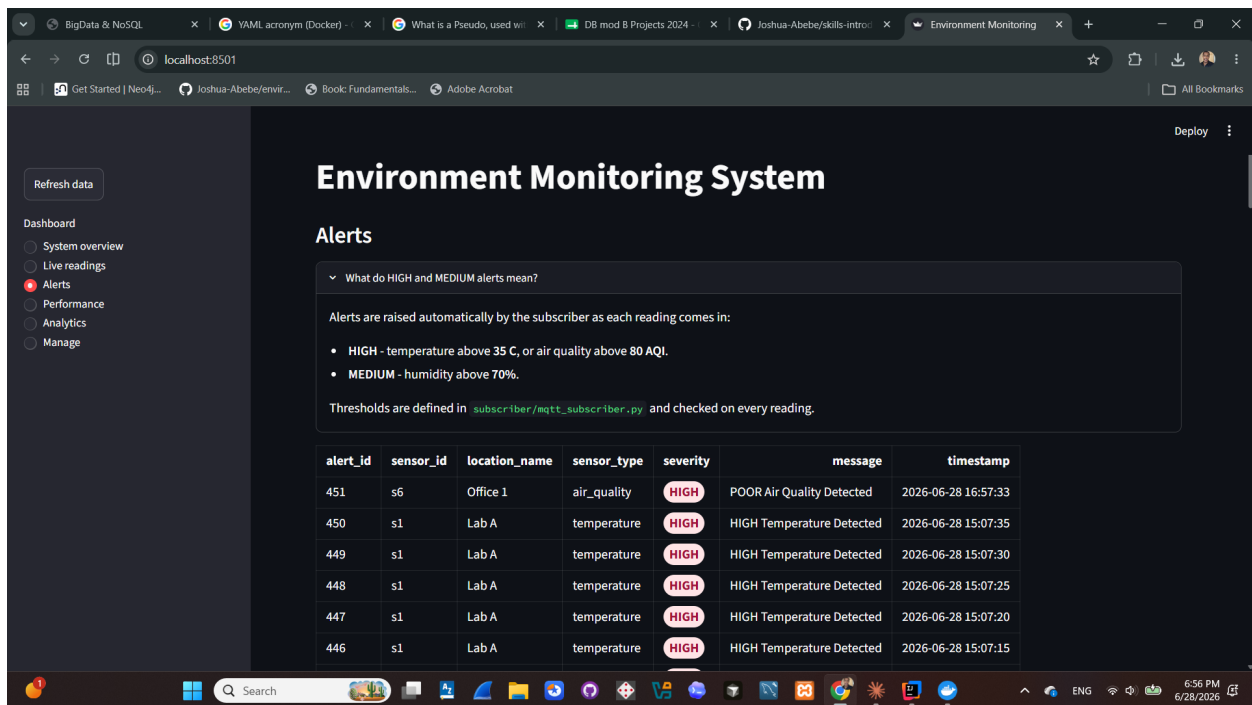
The screenshots below were captured from the running Docker Compose stack and show the live Streamlit dashboard.



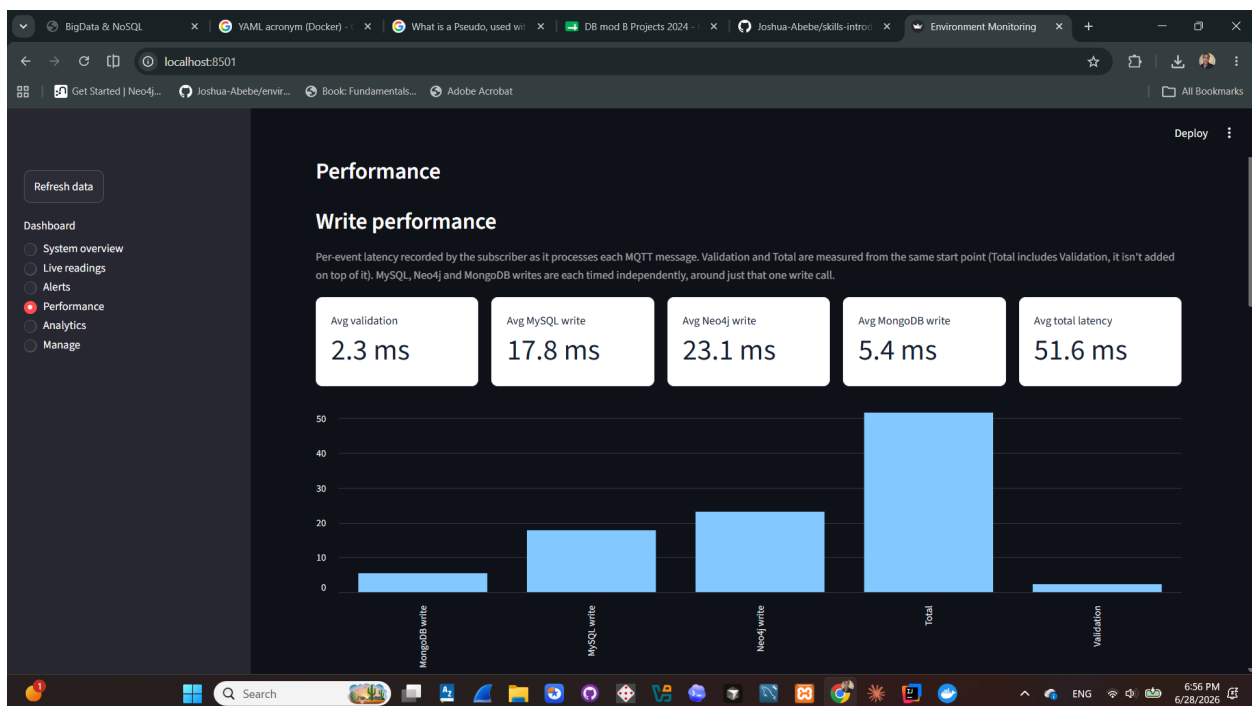
System overview — live counts of sensors, rooms, events recorded today and total alerts.



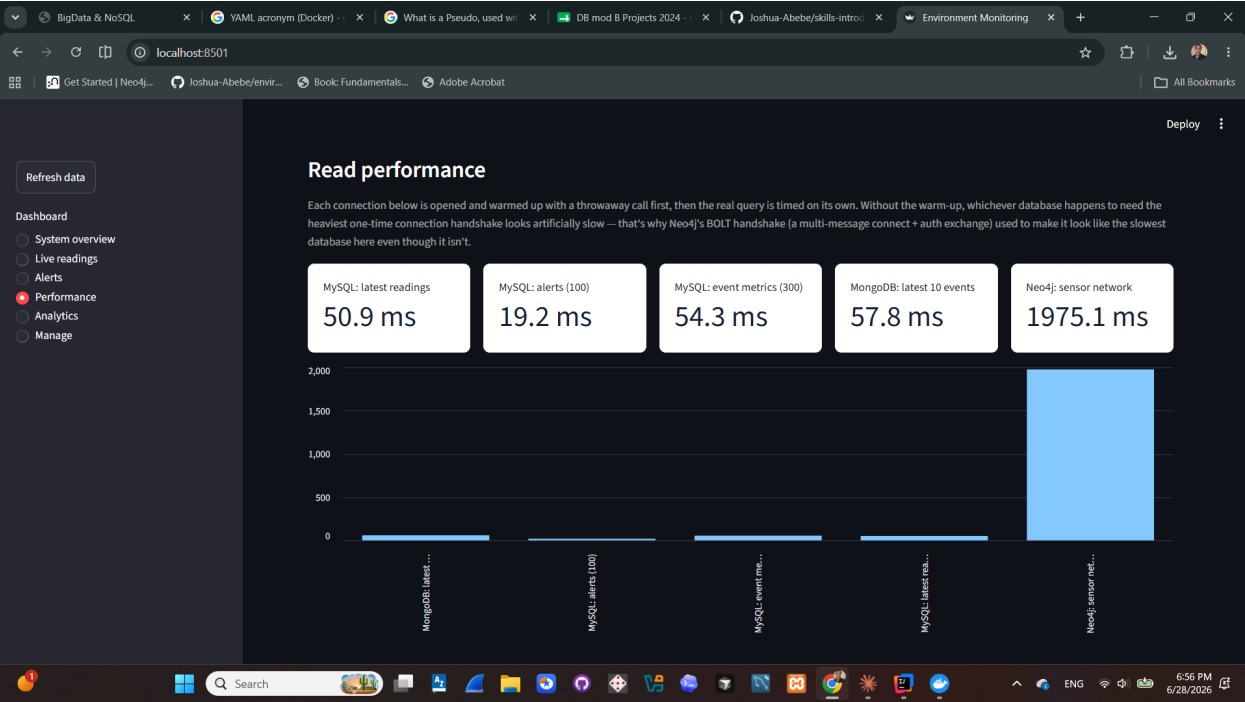
Live readings — most recent value per sensor, grouped by room, plus a recent-history table.



Alerts — severity-coded list of threshold excursions raised by the subscriber, with the threshold rules shown inline.



Performance — per-stage write latency (validation, MySQL, Neo4j, MongoDB, total) for every event processed.

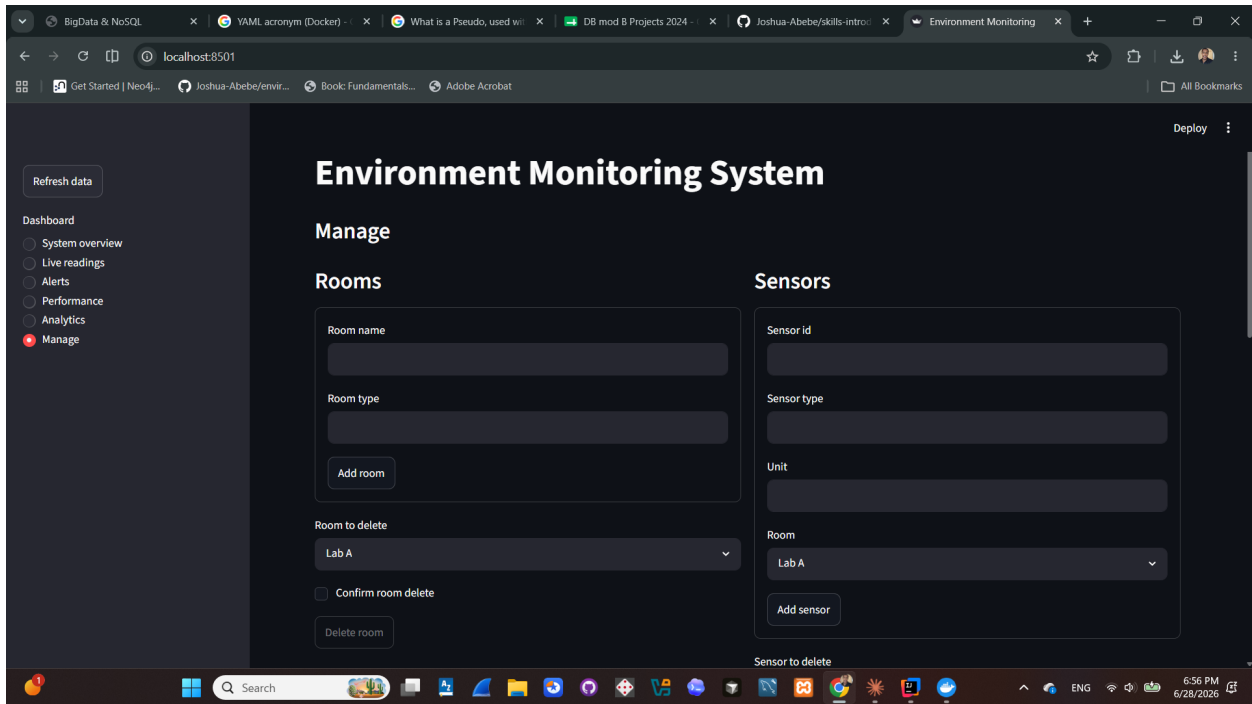


Performance — read-latency benchmark across MySQL, MongoDB and Neo4j, each warmed up before timing to keep the comparison fair.

The 'Environment Monitoring System' dashboard displays a table of analytics data. The table has seven columns: reading_id, sensor_id, sensor_type, location_name, value, unit, and timestamp. The data is organized into rows, showing various sensor readings from different locations. A sidebar on the left contains a 'Refresh data' button and a 'Dashboard' menu with options: System overview, Live readings, Alerts, Performance, Analytics (selected), and Manage. The top of the dashboard has a 'Deploy' button. The browser's address bar shows 'localhost:8501'.

reading_id	sensor_id	sensor_type	location_name	value	unit	timestamp
6237	s6	air_quality	Office 1	44.95	AQI	2026-06-28 18:56:38
6236	s4	temperature	Lab B	26.83	C	2026-06-28 18:56:37
6235	s1	temperature	Lab A	28.27	C	2026-06-28 18:56:36
6234	s5	humidity	Lab B	54.14	%	2026-06-28 18:56:34
6233	s2	humidity	Lab A	59.88	%	2026-06-28 18:56:33
6232	s4	temperature	Lab B	27.34	C	2026-06-28 18:56:32
6230	s3	air_quality	Server Room	40.68	AQI	2026-06-28 18:56:31
6231	s1	temperature	Lab A	27.82	C	2026-06-28 18:56:31
6229	s6	air_quality	Office 1	44.7	AQI	2026-06-28 18:56:29
6228	s5	humidity	Lab B	54.51	%	2026-06-28 18:56:27

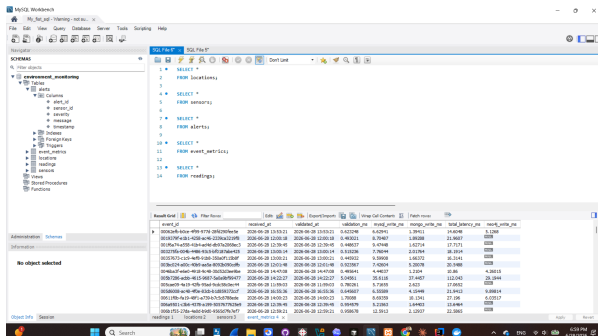
Analytics — tabbed cross-database views (latest readings shown; also averages, alerts, raw events, sensor/room network, statistics).



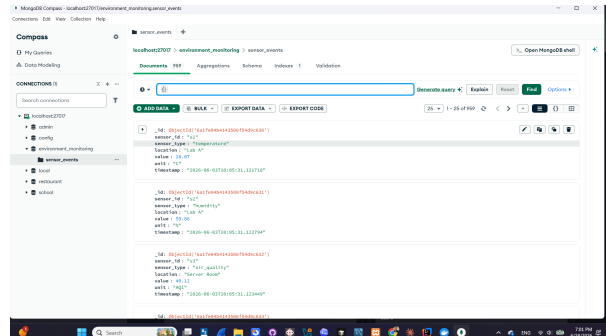
Manage — adding and removing rooms and sensors; the same page also creates Neo4j relationships between sensors, metrics and rooms.

Database Backends, Verified Directly

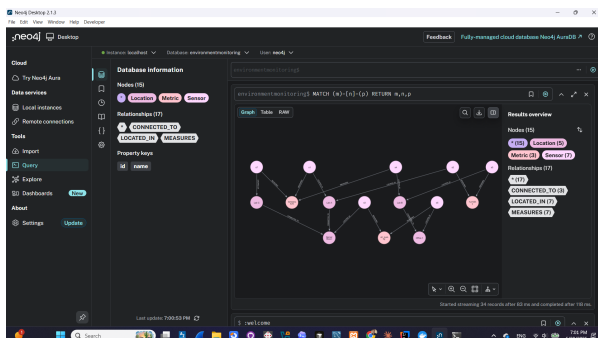
To confirm each store genuinely holds its own independent data, rather than the dashboard simply relaying one source, the native client for each database was also captured directly, alongside Docker Desktop showing all services running together.



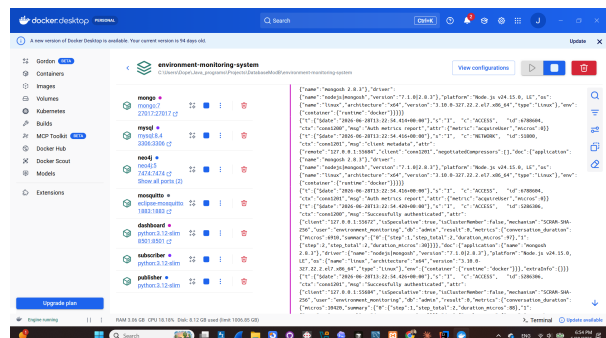
MySQL Workbench — event_metrics table (per-event pipeline timings).



MongoDB Compass — raw sensor_events collection.



Neo4j Browser — live sensor / location / metric graph.



Docker Desktop — all seven services running.

7. Conclusions and Future Work

The project delivers a working, fully containerized pipeline that ingests simulated environmental telemetry over MQTT and routes it in real time into three database paradigms chosen for their respective strengths, surfaced through a live multi-page dashboard. Along the way it demonstrates defensive event validation, edge-triggered alerting, per-database failure isolation, and per-stage performance instrumentation — all validated by an automated test suite that runs in CI on every change.

Possible future extensions include machine-learning-based anomaly detection in place of the fixed thresholds used here, deployment to a cloud environment, integration with real sensor hardware in place of the simulator, and splitting the shared Python image into per-service Dockerfiles so dependencies are not reinstalled on every container start.

8. Source Code

Full source code, configuration, automated tests and the CI workflow are available at: github.com/Joshua-Abebe/environment-monitoring-system. Setup and usage instructions are documented in the repository's README.